



Dart & Flutter



Write Once, Run Anywhere



What are Dart & Flutter

Dart is a programming language developed by Google. Flutter is a UI framework built on top of Dart.

Dart provides language features such as classes, null safety, async programming, and package management.

Flutter provides widgets, layouts, navigation, state management, and rendering.

In simple terms:

- Dart = the language
- Flutter = the framework
- Together they are used to build complete applications.

Think of Dart as the language/engine and Flutter as the car built on top of it:

- Dart: The underlying programming language that handles the logic, data, and compilation.
- Flutter: The toolkit full of pre-made UI buttons, layouts, and design components you use to build screens.

Applications

Flutter is an open-source framework created by Google to build cross-platform applications. Instead of writing separate code for Android (using Kotlin/Java), iOS (using Swift), and the Web (using HTML/JS), Flutter lets you write one single codebase using Dart that runs everywhere:

- Mobile: Android & iOS
- Desktop: Windows, macOS, & Linux
- Web: Web browsers

Good use cases:

- Business applications
- E-commerce applications
- Booking systems
- School projects
- Internal company tools
- Cross-platform mobile apps

Possible but less common:

- 2D games using Flame Engine
- Simple casual games

Dart vs. JavaScript / TypeScript

Dart vs JavaScript/TSX

- Similar async/await syntax
- Similar functional programming style
- TypeScript is very close to Dart
- Dart is more strongly structured

e.g.

JavaScript: `users.map(user => user.name)`

Dart: `users.map((user) => user.name)`

Dart vs. Python and Java

Dart vs Java

- Very similar syntax
- Both are object-oriented
- Dart has less boilerplate
- Dart has built-in null safety

Java: `public class Book { private String title; }`

Dart: `class Book { String title; Book(this.title); }`

Dart vs Python

- Python is dynamically typed
- Dart is statically typed
- Python uses indentation
- Dart uses braces

Python: `name = "Book"`

Dart: `String name = "Book";`

Example Project - BookStore

Here is a step-by-step roadmap to build a **Bookstore App using Dart & Flutter**.

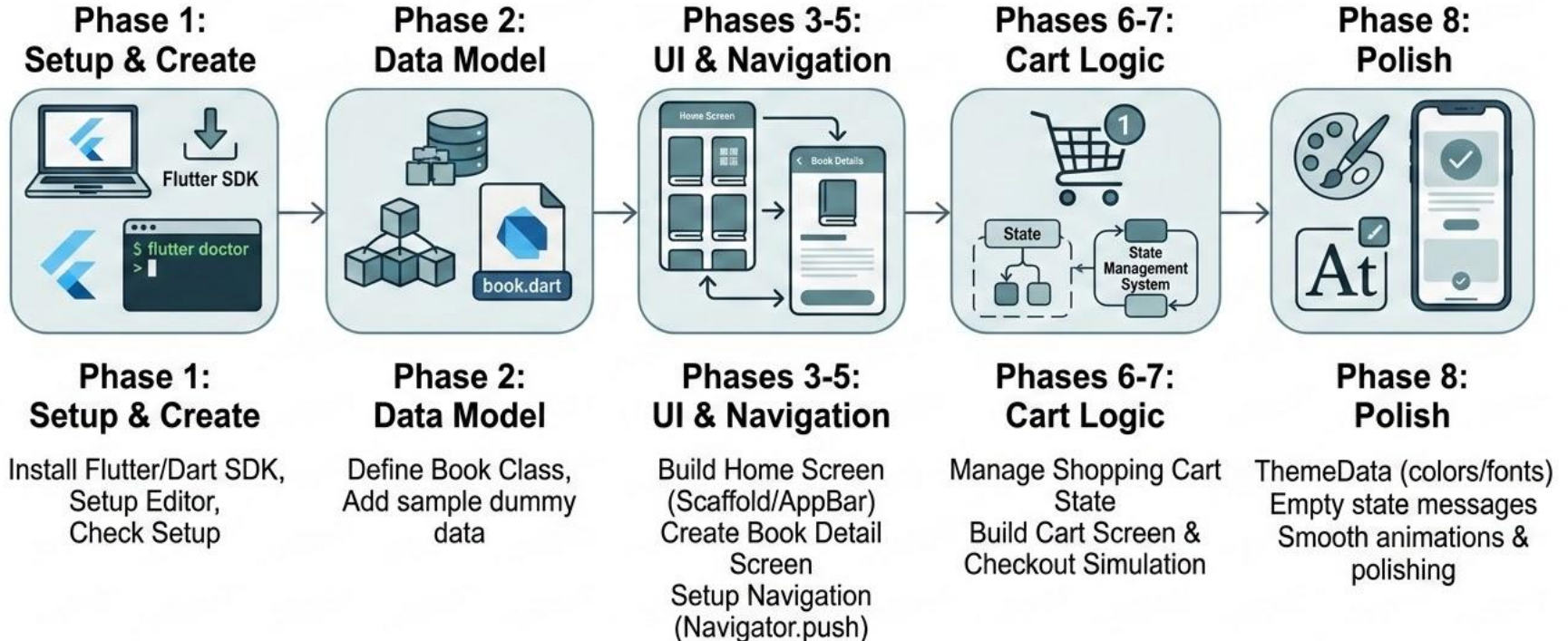
To keep things clear and non-technical, think of building a Flutter app like assembling Lego bricks: **Data (Dart)** is your foundation, **UI Widgets (Flutter)** are your visual blocks, and **State Management** is the system that makes everything interact when clicked.

Environment Setup

Goal: Prepare your computer for Flutter development.

1. **Install Flutter SDK:** Download Flutter from the official website. This includes the **Dart SDK** automatically.
2. **Setup Editor:** Download **VS Code** or **Android Studio**, and install the **Flutter** and **Dart** extensions.
3. **Check Setup:** Open your terminal and run: *flutter doctor*

Project Roadmap: Building the Bookstore App with Dart & Flutter



Phase 1: Create Project & Understand Folder Structure

Goal: Get your blank app running on a device or simulator.

Create a project by Bash or Terminal

```
flutter create bookstore_app
```

```
cd bookstore_app
```

Open the project in your editor.

Key folders to know:

- lib/: This is where all your Dart & Flutter code lives.
- lib/main.dart: The starting point (entry point) of your application.

Run the starter app (flutter run) to verify everything works.

Phase 2: Create the Book Data Model (Dart)

Goal: Define what a "Book" looks like in Dart code.

Create a file `lib/models/book.dart`:

Define a class `Book` with basic properties:

- `id (String)`
- `title (String)`
- `author (String)`
- `price (double)`
- `imageUrl (String)`
- `description (String)`

Add a list of sample dummy books in a mock data file so you have content to display later.

Phase 3: Build the Home Screen UI (Flutter Layout)

Goal: Display a list/grid of books on the screen.

Create a file `lib/screens/home_screen.dart`.

Use basic Flutter layout widgets:

- Scaffold: Provides the standard screen structure (Top bar, body background).
- AppBar: Shows the title ("Bookstore") and a shopping cart icon.
- GridView.builder or ListView.builder: Displays your list of books efficiently.

Create a custom widget `BookCard` to display individual book pictures, titles, and prices cleanly.

Phase 4: Build the Book Details Screen

Goal: Allow users to tap on a book to see full information.

Create `lib/screens/book_detail_screen.dart`.

Pass the selected Book object from the Home Screen to the Detail Screen.

Layout the page:

- Large book cover image at the top.
- Book title, author, and price in bold text.
- Full description text below.
- An "Add to Cart" button at the bottom.

Phase 5: Navigation Setup

Goal: Move smoothly between screens.

Use Flutter's built-in navigator (`Navigator.push`) when a user taps a BookCard.

Connect the "Back" button so users can return seamlessly from the Details screen to the Home screen.

Phase 6: Manage Shopping Cart State

Goal: Track which books the user wants to purchase.

Create a Cart model/service to store:

- A list of selected books.
- Functions to add a book, remove a book, and calculate the total price.

Pass this cart state around your app so the cart icon badge (e.g., showing 3 items) updates automatically across screens. (You can use simple `setState` to start, or lightweight packages like `provider`).

Phase 7: Build the Cart Screen & Checkout Simulation

Goal: Show selected books and let the user complete an order.

Create `lib/screens/cart_screen.dart`.

List all items currently in the cart with:

- Item name and price.
- A remove button (Trash icon) to delete items.
- Total sum displayed at the bottom.

Add a "Checkout" button that shows a success message (e.g., a simple pop-up dialog saying "Thank you for your order!") and clears the cart.

Phase 8: Polish and Styling

Goal: Make the app look modern and clean.

Configure custom colors and fonts in `main.dart` using `ThemeData`.

Add small details:

- Show an empty state message if the cart is empty ("Your cart is empty").
- Add smooth image loading placeholders.

References

Tutorials

[Dart tutorial](#)

[Dart Tutorial](#)

[Dart Tutorial - GeeksforGeeks](#)

[Learn Flutter](#)

[Flutter Tutorial - GeeksforGeeks](#)

Cheatsheets & project code

[manhtuandoan0410/Cheat_Sheets](#)